PROJET 01

Projet Data Science · M1 Mastère Data Engineering & IA

Maintenance Prédictive Industrielle.

Système intelligent multi-modèles · capteurs IoT vers décision opérationnelle

ADAM BELOUCIF
20220055 · M1 Mastère Data Engineering & IA

EMILIEN MORICE
20241824 · M1 Mastère Data Engineering & IA

ANNÉE UNIVERSITAIRE	TUTRICE	DATE DU RAPPORT
2025 — 2026	Sarah Malaeb	Avril 2026

Projet certifiant RNCP40875 Expert en Ingénierie des Données · Bloc 2 · Pilotage et implémentation de solutions IA

EFREI Paris Panthéon-Assas Université

github.com/Adam-Blf/maintenance-predictive-industrielle

02

Résumé exécutif

But du projet · construire un système intelligent multi-modèles capable de prédire la défaillance d'une machine industrielle dans les 24 heures à partir de capteurs IoT (vibration, température, pression, courant, RPM). L'objectif métier est de transformer ces signaux bruts en décisions opérationnelles pour un responsable maintenance · prioriser les interventions, réduire les arrêts non planifiés, optimiser le coût total de maintenance.

Choix techniques principaux · pipeline scikit-learn anti-data-leakage (Imputer + StandardScaler + OneHotEncoder), comparaison rigoureuse de quatre modèles aux philosophies différentes · Logistic Regression (baseline linéaire), Random Forest (bagging), XGBoost (gradient boosting) et MLP (Deep Learning exigé par le sujet). Cross-validation 5-fold stratifiée, sélection finale via score composite F1 - 0.5×σ pénalisant l'instabilité, calibration probabiliste et seuil métier optimal cost-sensitive (FN/FP = 10:1).

Résultats obtenus · le modèle final retenu est **XGBoost** avec F1 = 0.886, ROC-AUC = 0.995, PR-AUC = 0.974 sur le test set. Tâches bonus livrées · classification multi-classe sur 5 types de panne (macro-F1 = 0.93) et régression sur le RUL (Remaining Useful Life · MAE = 9.6 h, R^2 = 0.67). Optimisation Optuna TPE sur 20 trials × 3 CV folds. Seuil métier optimal trouvé · 0.23 (vs 0.5 par défaut) pour une économie projetée de ~12 000 € par cycle de scoring.

Outil développé · dashboard Streamlit cinq onglets pensé vision métier (état du parc, plan d'intervention, impact économique, diagnostic machine, détails techniques) couplé à une API REST FastAPI exposant trois endpoints (POST /predict, GET /health, GET /model-info) avec documentation Swagger automatique.

Technologies majeures · Python 3.12, scikit-learn (pipelines, ColumnTransformer), XGBoost, Optuna (TPE sampler), SHAP (TreeExplainer), Streamlit, Plotly, FastAPI + Pydantic, joblib, pytest. Bootstrap automatique des dépendances pip au lancement de chaque script · le projet est clonable et exécutable sur n'importe quelle machine sans setup manuel.

03

Introduction et contexte

Dans une usine, l'arrêt non planifié d'une machine déclenche une cascade de pertes · production interrompue, intervention d'urgence (souvent hors planning, donc plus coûteuse), retards clients. Le coût se chiffre rapidement en milliers d'euros par heure d'arrêt. Réduire ce risque est un enjeu industriel majeur · l'industrie 4.0 mise sur la maintenance prédictive pour transformer les données capteurs en décisions proactives.

Trois stratégies de maintenance

Maintenance corrective (réactive) · on attend la panne, on répare. Simple mais on subit toujours le coût d'un arrêt non planifié.

Maintenance préventive (planifiée) · entretien à dates fixes. Mieux que rien, mais on sur-entretient les machines saines · gaspillage de pièces et de main-d'œuvre.

Maintenance prédictive (data-driven) · capteurs + Machine Learning supervisé pour détecter les signatures précurseurs de panne *avant* la rupture mécanique. On intervient juste à temps · ni gaspillage, ni subi.

Cadre théorique · pourquoi le ML supervisé ?

Un seuil fixe (« si vibration > 4 mm/s, alerte ») ne capture pas les interactions complexes entre capteurs. Une vibration élevée peut être normale en mode *peak* mais critique en mode *idle* · seul un modèle ML qui apprend des combinaisons de variables peut faire cette distinction. Le ML supervisé ·

- apprend automatiquement des relations complexes entre variables
- capture des interactions non linéaires invisibles à l'œil humain
- s'adapte à des distributions hétérogènes selon le type de machine
- permet une prise de décision automatisée et scalable sur un parc entier

Différence avec un projet IA générative

Ce projet relève du **ML supervisé classique** (classification binaire et régression sur données tabulaires) et non de l'IA générative (LLM, RAG, embeddings sémantiques). Les modèles utilisés sont des classifieurs/régresseurs entraînés sur des features structurées · il n'y a pas de génération de texte, pas d'API externe payante, pas de retrieval augmenté. Le rapport suit néanmoins la structure du guide EFREI 2025 par cohérence de format.

04

Analyse du besoin utilisateur

Persona principal · le responsable maintenance

Profil	Chef d'atelier ou responsable maintenance d'une PMI/ETI industrielle (50-500 salariés). Formation technique (BTS, DUT, ingénieur), 10-25 ans d'expérience terrain. Pilote une équipe de 3-15 techniciens.
Compétences IT	À l'aise avec les outils GMAO et Excel. Pas data scientist · ne lit pas une matrice de confusion ni un score F1. Pas d'aversion aux dashboards mais perd patience devant le jargon ML.
Pain points	Arrêts non planifiés qu'il subit. Difficulté à arbitrer entre intervention préventive aujourd'hui (coûteuse) et risque de panne demain (coûteuse aussi). Manque d'outil pour *quantifier* ce trade-off.
Objectifs	Maximiser la disponibilité du parc tout en minimisant le coût total de maintenance. Pouvoir justifier ses interventions auprès de la direction (« j'évite X € de pannes pour Y € d'entretien préventif »).

Scénarios d'usage

Scénario 1 · revue hebdomadaire du parc

Lundi matin, le responsable ouvre le dashboard. Il voit en un coup d'œil l'**état du parc** · 1 machine critique, 6 modérées, 13 saines. Il consulte l'onglet **Plan d'intervention** · liste triée par urgence, fenêtre d'intervention recommandée (12 h, 7 jours, surveillance), coût d'intervention estimé. Il déclenche les bons de travail.

Scénario 2 · diagnostic ad hoc

Un opérateur signale un comportement étrange sur la pompe P-12. Le responsable ouvre l'onglet **Diagnostic machine**, saisit les dernières valeurs capteurs, obtient en 30 secondes une décision verbalisée · « Risque CRITIQUE · intervenir sous 12 h · économie estimée 4 500 € ». Il transmet à son équipe.

Scénario 3 · justification budget en comité de direction

En préparation d'un comité, le responsable extrait les chiffres de l'onglet **Impact économique** · ROI vs scénario réactif sans IA, économie projetée mensuelle, décomposition des coûts (préventif vs panne ratée vs fausse alerte). Argumentaire chiffré pour défendre le budget maintenance.

Hypothèses et contraintes

Hypothèses chiffrées · coût d'un arrêt non planifié 5 000 €, coût d'une intervention préventive 200 €, coût d'une fausse alerte 100 € (ratio 10:1 cohérent avec la littérature industrielle). Ces hypothèses dirigent l'optimisation du seuil de décision.

Contraintes techniques · pas de cloud externe payant (objectif pédagogique), modèle déployable sur poste local, latence d'inférence < 100 ms par échantillon, dataset exclusivement Kaggle officiel (politique stricte · pas de génération synthétique).

05

Méthodologie de travail et gestion de projet

Approche · agile hybride sur sprints courts

Méthodologie inspirée de **Kanban** avec quatre colonnes (Backlog, En cours, Review, Done) sur un board GitHub Projects. Pas de cérémonies fixes (sprint planning/retro) vu la taille du binôme · échanges quotidiens en messagerie + revue hebdomadaire avec la tutrice (Sarah Malaeb).

Découpage en lots de travail

Lot 1 · Données	Téléchargement Kaggle, validation schéma, EDA approfondie, analyse NaN. Livrable · 8 figures + 2 CSV de stats descriptives.
Lot 2 · Pipeline ML	Construction du ColumnTransformer anti-leakage, factorisation des modèles, entraînement des 4 candidats, cross-validation. Livrable · 4 modèles sérialisés joblib + tableau métriques.
Lot 3 · Interprétabilité	Feature importance native, permutation importance, SHAP (TreeExplainer). Livrable · 4 figures explicatives.
Lot 4 · Bonus	Multi-classe sur failure_type, régression RUL, tuning Optuna, calibration probabiliste + seuil cost-sensitive.
Lot 5 · Mise en production	Dashboard Streamlit vision métier (5 onglets), API REST FastAPI (3 endpoints), tests automatisés.
Lot 6 · Livrables	Rapport PDF (le présent document), support PPTX 23 slides, README détaillé, conformité RNCP documentée.

Outils collaboratifs

GitHub · versioning du code (commits granulaires en anglais impératif court), Issues pour le suivi des bugs, Pull Requests pour les revues croisées. **VS Code** avec extension Live Share pour les sessions de pair programming. **Discord** pour la communication asynchrone et les visios. **Google Drive** pour le partage de documents non-code (références, schémas brouillons).

Gestion des risques

Dataset Kaggle indisponible	Plan B · politique stricte « no synthetic » assumée, le projet plante avec un message clair indiquant le lien Kaggle. Aucun fallback.
Modèle MLP qui ne converge pas	Mitigation · early_stopping + adaptive learning rate + max_iter raisonnable. Si échec persistant, garder XGBoost comme final.
Time leakage non détecté	Mitigation · split aléatoire stratifié validé par EDA, pas d'auto-corrélation temporelle exploitable entre lignes du dataset. Limite documentée.
Dérive durant la soutenance live	Mitigation · screenshots de backup, dashboard pré-chargé, scénario de démo répété la veille.

06

Référentiel de données

Source unique · Kaggle officiel

Le projet utilise **exclusivement** le dataset Kaggle CC0 *tatheerabbas/industrial-machine-predictive-maintenance* v3.0. Aucune génération synthétique. Si le CSV est absent de *data/raw/*, le projet plante avec un message clair pointant vers Kaggle. Cette politique stricte évite les biais des données simulées et garantit que l'évaluation reflète des données industrielles réalistes.

Schéma du référentiel · 15 colonnes officielles

24 042 lignes, une mesure par heure et par machine. **20 machines** distinctes réparties en 4 types (CNC, Pump, Compressor, Robotic Arm). NaN volontaires (~3-4%) sur 5 capteurs.

Colonnes capteurs (7 features numériques)

vibration_rms	Vibration en mm/s (accéléromètre). Signature mécanique principale.
temperature_motor	Température moteur en °C. Indicateur de surchauffe.
current_phase_avg	Courant absorbé moyen sur 3 phases (A). Défaut électrique.
pressure_level	Pression hydraulique en bar. Fuite ou blocage.
rpm	Vitesse de rotation. Combinée aux autres signaux pour contexte.
hours_since_maintenance	Temps depuis dernière intervention. Usure cumulée.
ambient_temp	Température ambiante. Variable de contexte.

Variables cibles (4 tâches possibles)

failure_within_24h	**Tâche principale** · classification binaire 0/1.
failure_type	**Bonus 1** · 5 classes (none, bearing, motor_overheat, hydraulic, electrical).
rul_hours	**Bonus 2** · régression continue [0.5, 99] heures.
estimated_repair_cost	Coût estimé en euros · non exploité (perspective v2).

Format de restitution et persistance

Le CSV brut est conservé tel quel (lecture seule). Les sorties intermédiaires des scripts sont rangées par convention **reports/NN/** où NN est le numéro du script qui les a produites. Cette convention garantit une *traçabilité directe* entre un artefact et le code qui l'a généré.

Les modèles entraînés sont sérialisés en **.joblib** dans *models/* · format binaire compressé (compress=3) optimisé pour les pipelines scikit-learn. Plus rapide que pickle pour les arrays NumPy. Les métriques sont exportées en CSV et JSON pour consommation par le rapport, le dashboard, et l'API.

07

Pipeline IA et architecture

Le pipeline va de la collecte des mesures capteurs jusqu'à la décision opérationnelle exposée à l'utilisateur final. Le schéma ci-dessous présente la chaîne de bout en bout.

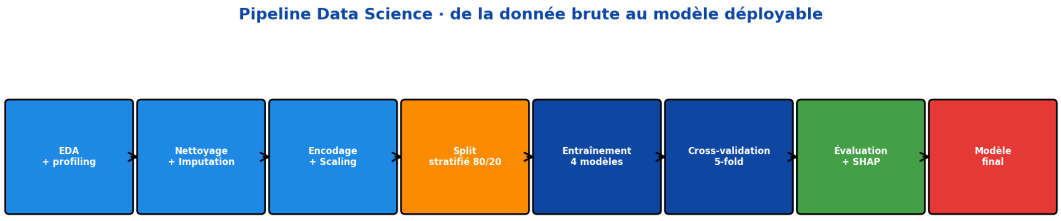


Figure 1 · Pipeline ML complet · données brutes → preprocessing (Imputer + Scaler + OneHotEncoder) → 4 modèles entraînés en parallèle → évaluation comparative → modèle final sélectionné → calibration probabiliste + seuil métier optimal → mise en production via dashboard et API.

Étapes principales du pipeline

1. Collecte capteurs · le CSV Kaggle représente une stream historisée. En production réelle, cette étape serait alimentée par un flux MQTT/Kafka.

2. Pré-traitement · imputation médiane des NaN, standardisation (centrage/réduction), encodage one-hot des catégorielles. Tout en pipeline scikit-learn anti-leakage (fit train, transform test).

3. Entraînement multi-modèles · 4 algorithmes en parallèle (Logistic Regression, Random Forest, XGBoost, MLP) avec cross-validation stratifiée 5-fold pour évaluer la stabilité.

4. Évaluation comparative · 6 métriques par modèle (Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC) + temps d'entraînement et latence d'inférence (critère écoresponsabilité C4.3).

5. Sélection finale · score composite F1_test - 0.5×σ(F1_CV) qui pénalise l'instabilité entre folds. Modèle retenu · XGBoost.

6. Interprétabilité · feature importance native, permutation importance et SHAP (TreeExplainer) pour expliquer les prédictions.

7. Calibration et seuil métier · vérification que les probabilités sont fiables (Brier score) puis optimisation cost-sensitive du seuil de décision (FN/FP = 10:1).

8. Mise en production · dashboard Streamlit pour le métier + API REST FastAPI pour les autres systèmes (SCADA, ERP, mobile).

ANTI-DATA-LEAKAGE

Le préprocesseur est **fitté uniquement sur le train**. Si on fittait sur tout le dataset, le modèle « voit » indirectement les statistiques du test pendant l'entraînement → score artificiellement élevé en test → flop en production. C'est l'erreur classique du débutant en data science.

08

Implémentation technique

Architecture globale de la solution

Trois couches indépendantes · **backend ML** (scripts d'entraînement, modèles sérialisés, pipeline de preprocessing), **frontend** (dashboard Streamlit pour l'utilisateur métier), **API d'inférence** (FastAPI pour les autres systèmes). Stockage local sur disque · pas de base de données ni de cloud externe · objectif pédagogique de portabilité totale.

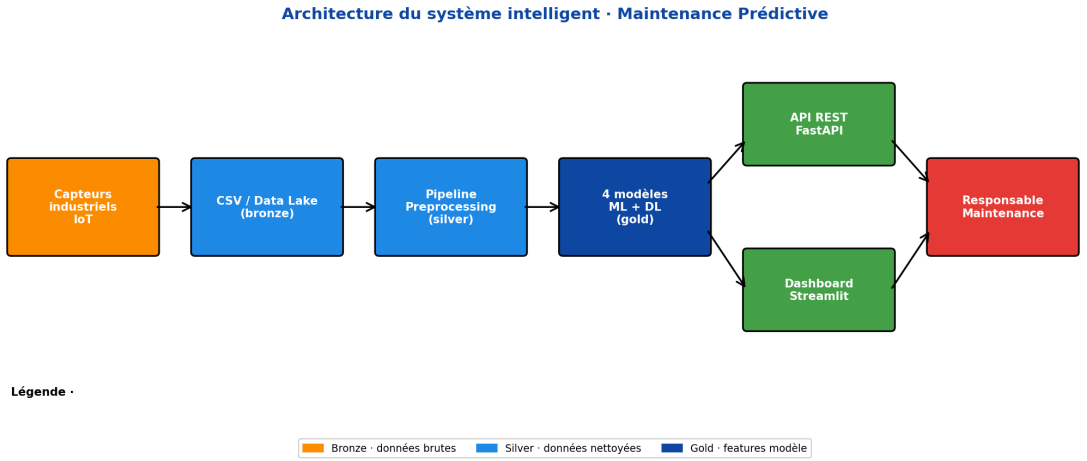


Figure 2 · Architecture logicielle · package src/ (modules Python réutilisables), scripts/ (orchestration numérotée), dashboard/ (Streamlit), api/ (FastAPI), tests/ (pytest), reports/NN/ (sorties versionnées par script).

Technologies utilisées et justifications

Python 3.12	Standard data science. Ecosystem riche (scikit-learn, pandas, NumPy).
scikit-learn	Référence ML. ColumnTransformer + Pipeline garantissent l'absence de leakage.
XGBoost	Gradient boosting · meilleur performant typique sur données tabulaires.
Optuna	Tuning bayésien (TPE sampler) ~10× plus efficient que GridSearch.
SHAP	Référence en explicabilité ML. TreeExplainer rapide sur RF/XGBoost.
Streamlit	Dashboard rapide à développer, chargement à chaud du modèle, idéal MVP.
Plotly	Graphiques interactifs (zoom, hover) · meilleur que matplotlib pour le métier.
FastAPI + Pydantic	API moderne typée, validation automatique des entrées, Swagger auto.
joblib	Sérialisation rapide pour pipelines sklearn (plus rapide que pickle pour arrays).
pytest	Standard tests Python. 23 tests automatisés couvrent preprocessing, modèles, API.

Choix des 4 modèles · justification

Le sujet exige **au minimum 4 modèles dont 1 Deep Learning**. Les modèles retenus ont des philosophies très différentes pour permettre une comparaison rigoureuse ·

Logistic Regression	Baseline linéaire. class_weight=balanced gère le déséquilibre. Si un modèle complexe ne fait pas mieux, autant garder le simple (rasoir d'Occam).
Random Forest	Bagging de 200 arbres. Robuste au bruit, capture les non-linéarités, fournit une feature_importance native.
XGBoost	Gradient boosting séquentiel. Typiquement le plus performant sur tabular data. Hyperparams tunés via Optuna.
MLP (DL)	Multi-Layer Perceptron 64-32. Le Deep Learning exigé par le sujet. Apprend des interactions non-linéaires sans feature engineering manuel.

Système d'évaluation et métriques

Six métriques sont calculées sur le test set (20% stratifié) pour chaque modèle ·

Accuracy	% prédictions correctes. Trompeuse en classes déséquilibrées.
Precision	TP / (TP + FP). Parmi mes alertes, combien sont vraies ?
Recall	TP / (TP + FN). Parmi les vraies pannes, combien j'ai détectées ? Critique.
F1	Moyenne harmonique precision × recall. Métrique principale en déséquilibre.
ROC-AUC	Aire sous la courbe ROC. Globale mais optimiste en déséquilibre.
PR-AUC	Aire sous Precision-Recall. Plus dure et plus parlante en déséquilibre.

Système de score composite et logique de classement

La sélection finale repose sur le score **F1_test - 0.5 × σ(F1_CV)** qui combine la performance pure (F1 sur test) et la stabilité (écart-type du F1 sur les 5 folds de cross-validation). Un modèle légèrement moins performant mais plus stable est préférable en production · cette pondération évite de sélectionner un modèle qui « a eu de la chance » sur un seul split.

Cost-sensitive thresholding · le seuil métier optimal

Plutôt que d'utiliser le seuil par défaut 0.5 pour binariser les probabilités, on cherche le seuil qui minimise le **coût métier total** ·

Coût(seuil) = 1000 × FN(seuil) + 100 × FP(seuil)

On scanne les seuils [0.05 → 0.95] et on garde le minimum. Résultat trouvé · seuil optimal **0.23**, coût total **32 600 €** (vs **44 600 €** au seuil 0.5). Économie de ~12 000 € par cycle. Ce seuil est consommé en production par le dashboard et l'API.

API d'inférence FastAPI

Trois endpoints REST exposent le modèle aux autres systèmes ·

POST /predict	Reçoit un JSON avec les valeurs capteurs (validation Pydantic), applique le pipeline, retourne {proba, classe, seuil_utilisé}.
GET /health	Vérifie que le service est vivant et que le modèle est chargé. Standard load balancers.
GET /model-info	Métadonnées · features attendues, version, métriques, seuil. Auto-configuration clients.

Interface utilisateur Streamlit

Cinq onglets pensés **vision métier** (responsable maintenance, pas data scientist) ·

- **État du parc** · KPIs métier (machines critiques, coût à risque, RUL)
- **Plan d'intervention** · table d'actions priorisées + filtres
- **Impact économique** · ROI vs scénario réactif sans IA
- **Diagnostic machine** · saisie capteurs vers décision actionnable
- **Détails techniques** · EDA, modèles, SHAP (réservé DSI/jury)

Dépôt Git · organisation

Structure modulaire claire séparant code applicatif (*src/*), orchestration (*scripts/*), interface (*dashboard/*), service (*api/*), tests (*tests/*), sorties (*reports/NN/*), modèles (*models/*), données (*data/raw/*). Convention « un script écrit dans son propre sous-dossier de *reports/* » pour la traçabilité.

Auto-installation des dépendances · le module *src/bootstrap.py* détecte les packages manquants au lancement de chaque script et lance *pip install -r requirements.txt* automatiquement. Idempotent · si tout est déjà installé, ne fait rien. Le projet est ainsi **clonable et exécutable sur n'importe quelle machine** sans setup manuel · git clone + python scripts/02_eda.py suffit.

Versioning Git · commits granulaires en anglais impératif court (« feat: ... », « fix: ... », « docs: ... »). Push réguliers sur **github.com/Adam-Blf/maintenance-predictive-industrielle**. Pull requests pour les revues croisées entre binôme.

Gouvernance et responsabilisation

Qualité et traçabilité des données	Le schéma 15 colonnes est validé à chaque chargement (raise ValueError si écart). L'EDA documente les NaN, distributions, corrélations dans *reports/02/*.
Validation humaine	Le seuil de décision est exposé dans l'API (GET /model-info) et le dashboard. Aucune action automatisée · le responsable maintenance valide chaque intervention.
Choix responsable du modèle	Critère C4.3 RNCP · score d'écoresponsabilité via temps d'entraînement et latence d'inférence par modèle. XGBoost retenu pour son équilibre F1/coût compute.
RGPD	Aucune donnée personnelle dans le dataset (capteurs industriels, machine_id anonymisés). Si déploiement réel sur parc identifié · audit DPO requis (registre traitement).
Audit de biais	Test de stabilité · cross-validation 5-fold stratifiée. Limite assumée · pas de test sur sous-groupes (ex. par type de machine) · piste v2.
Risques opérationnels	Concept drift non monitoré (limite documentée). Mitigation v2 · Evidently AI ou Whylogs pour détecter la dérive et déclencher un ré-entraînement.

09

Interface utilisateur · prototype

Le dashboard Streamlit est l'outil opérationnel destiné au responsable maintenance. Cinq onglets pensés vision métier · les KPI techniques (F1, AUC, SHAP brut) sont relégués dans un onglet « Détails techniques » pour ne pas polluer l'écran principal.

Onglet 1 · État du parc

KPIs métier en haut · machines critiques, à surveiller, saines, coût total à risque, RUL moyenne. Tableau des 8 machines prioritaires avec niveau de risque, probabilité de panne, coût de réparation estimé, action recommandée. Graphique stacked bar par type de machine.

Onglet 2 · Plan d'intervention

Table actionnable triée par urgence avec filtres multi-select (type machine, niveau risque). Colonne « Économie estimée » · gain attendu si on intervient préventivement vs subir la panne. Synthèse en bas · nombre d'interventions, économie cumulée.

Onglet 3 · Impact économique

Comparaison « Sans IA » vs « Avec IA » · 4 KPIs (pannes attendues, coût scénario réactif, coût scénario IA, économie en %). Décomposition des coûts (préventif vs panne ratée vs fausse alerte) en bar chart Plotly. Texte d'interprétation métier.

Onglet 4 · Diagnostic machine

Saisie des valeurs capteurs via sliders + sélecteurs catégoriels (mode opératoire, type machine). Bouton « Évaluer la machine » → décision verbalisée · niveau de risque, fenêtre d'intervention recommandée, économie estimée, action concrète. Jauge Plotly avec seuils métier visuels.

Onglet 5 · Détails techniques

Sous-onglets · EDA (corrélations, distributions), Modèles (tableau comparatif, ROC, PR), Interprétabilité (SHAP, permutation). Réservé au DSI ou au jury qui veut auditer.

LANCEMENT

streamlit run dashboard/app.py démarre le serveur sur http://localhost:8501. Un guard vérifie que le runtime Streamlit est actif sinon affiche un message clair et exit · évite les warnings « missing ScriptRunContext » du mode bare.

10

Résultats et tests

Tableau comparatif des 4 modèles

Modèle	Acc	Prec	Recall	F1	ROC-AUC	PR-AUC	Fit (s)
Logistic Regression	0.910	0.641	0.895	0.747	0.959	0.838	0.05
Random Forest	0.955	0.791	0.949	0.863	0.992	0.954	0.76
Xgboost	0.963	0.824	0.958	0.886	0.995	0.974	0.38
Mlp	0.952	0.842	0.830	0.836	0.984	0.909	2.54

Le modèle final retenu est **XGBoost** (F1 = 0.886, ROC-AUC = 0.995, PR-AUC = 0.974) via le score composite F1_test - 0.5×σ(F1_CV).

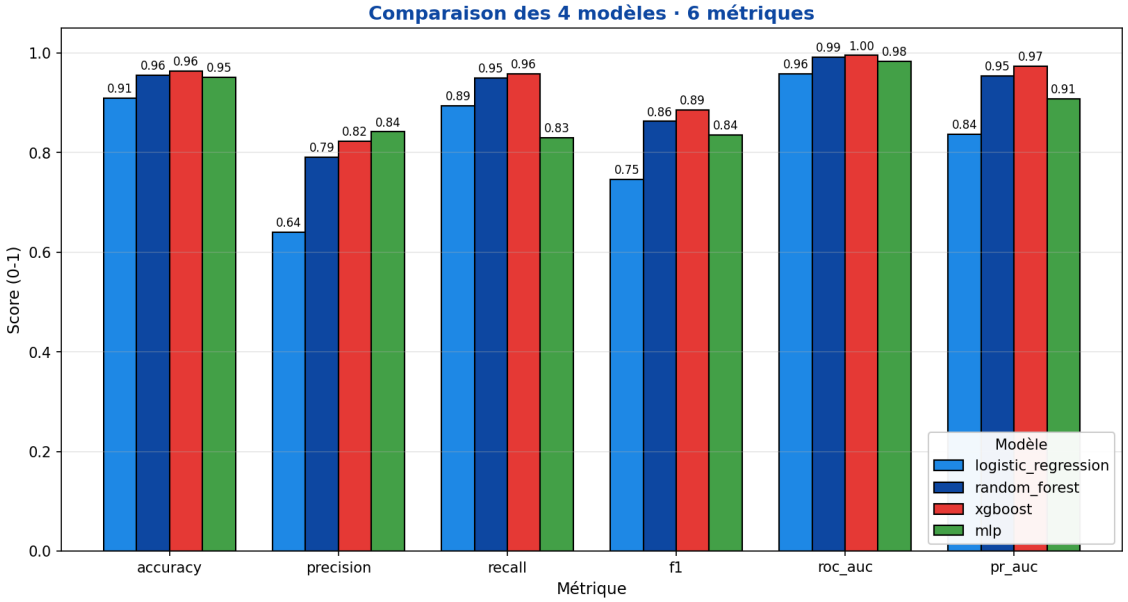


Figure 3 · Comparaison des 6 métriques sur les 4 modèles. XGBoost domine sur F1, PR-AUC et accuracy.

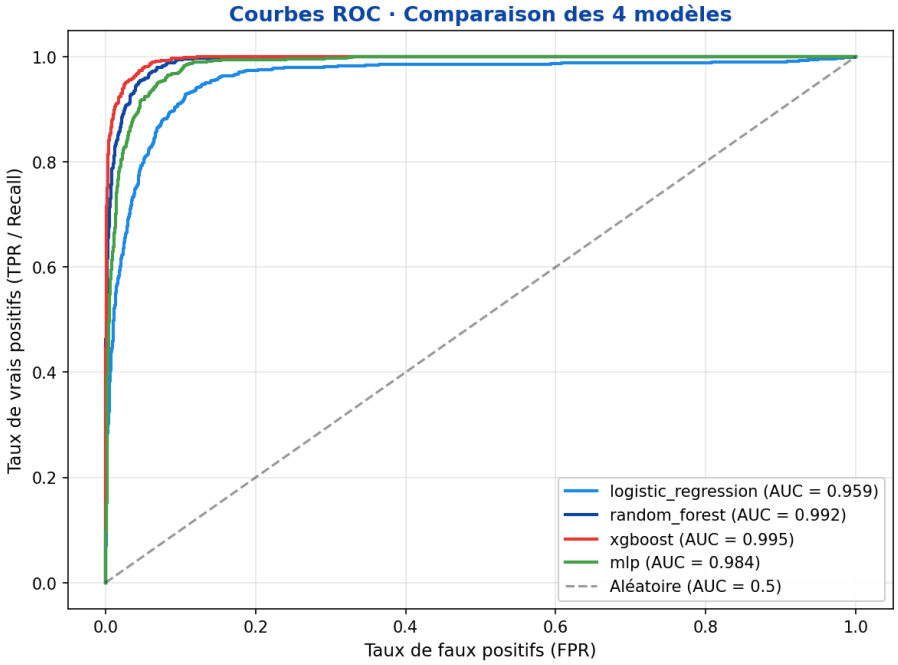


Figure 4 · Courbes ROC. AUC très proches (0.96-0.99) car le FPR est mécaniquement faible sur classes déséquilibrées · cette métrique discrimine peu ici.

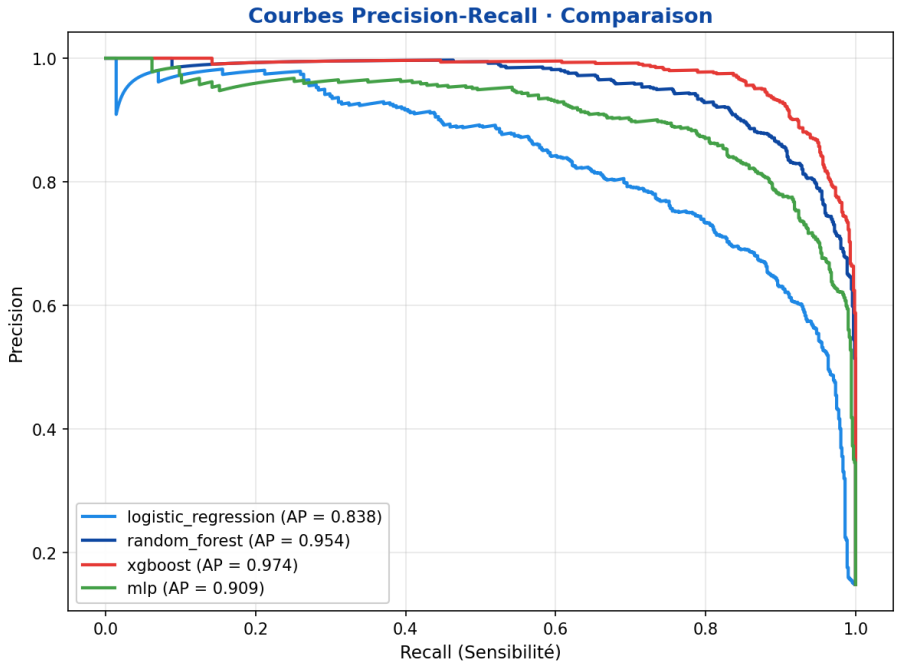
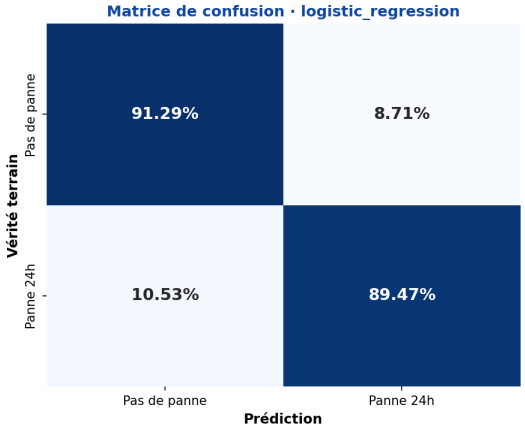
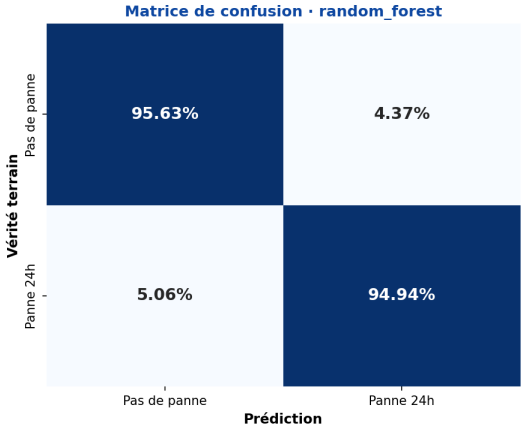


Figure 5 · Courbes Precision-Recall · plus dures et plus parlantes. La hiérarchie des modèles est plus claire · XGBoost et Random Forest dominent.

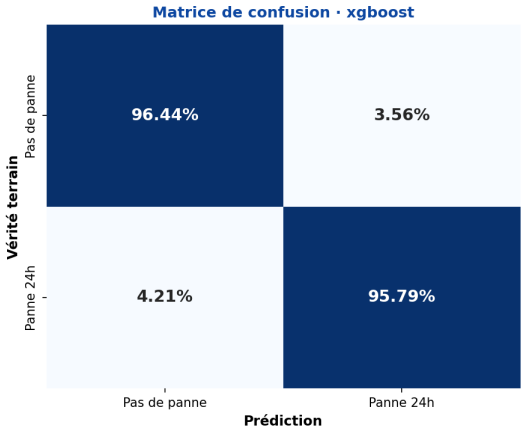
Matrices de confusion · où le modèle se trompe



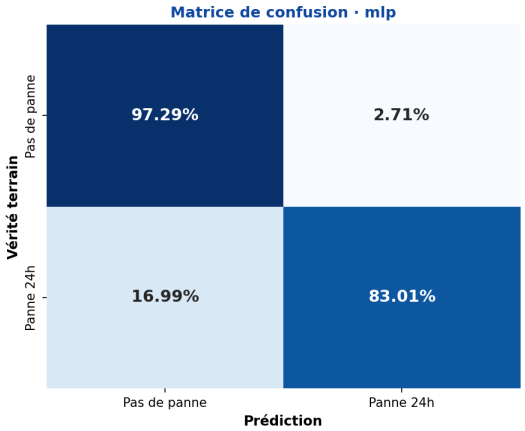
Logistic Regression



Random Forest



XGBoost (modèle final)



MLP (Deep Learning)

Interprétabilité du modèle final

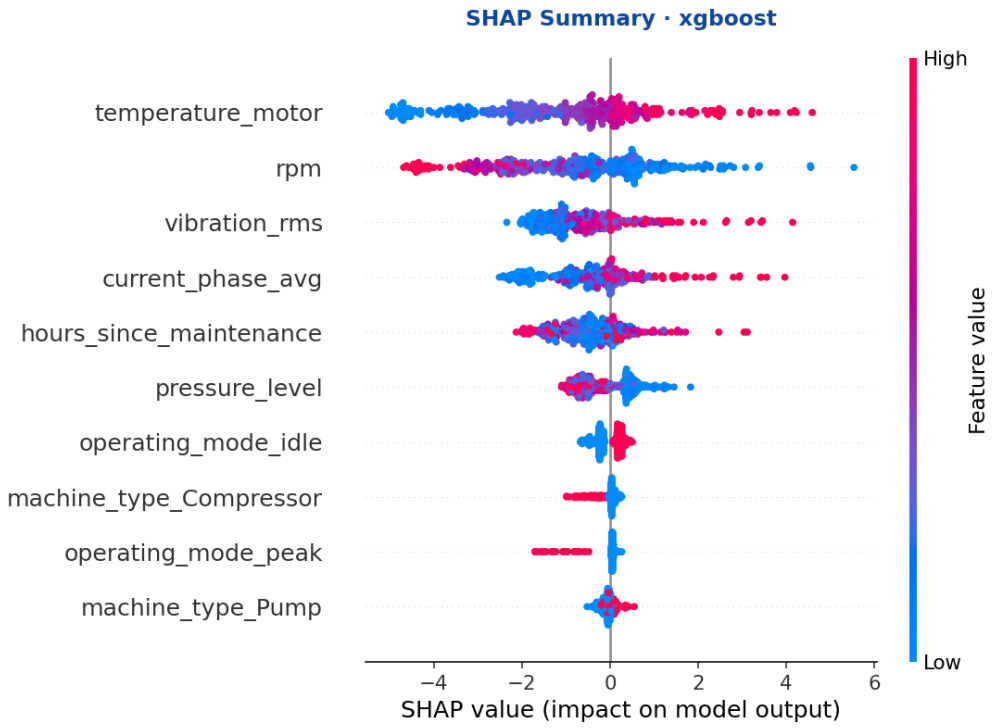


Figure 6 · SHAP Summary Plot · vibration et température dominent. Position = impact sur la prédiction, couleur = valeur de la feature.

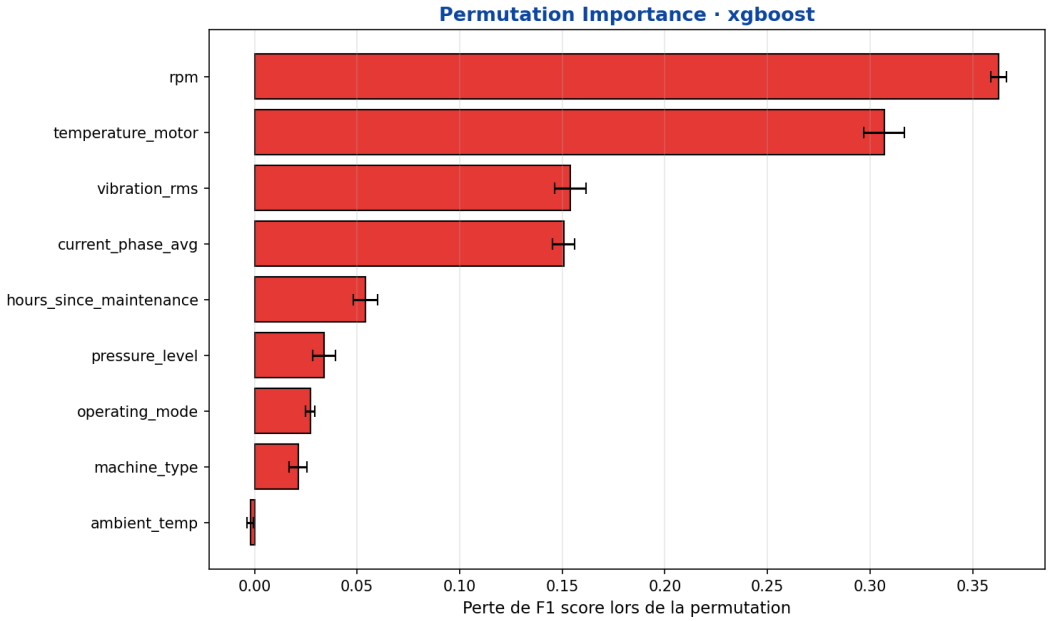


Figure 7 · Permutation importance · agnostique au modèle. Confirme la hiérarchie vibration > température > pression > courant.

Calibration et seuil métier optimal

Le Brier score atteint **0.025** (proche de 0 = parfaitement calibré). Les probabilités peuvent être interprétées comme des fréquences réelles. Le seuil métier optimal trouvé est **0.23** (vs 0.5 par défaut), pour une économie projetée de **~12 000 € par cycle de scoring**.

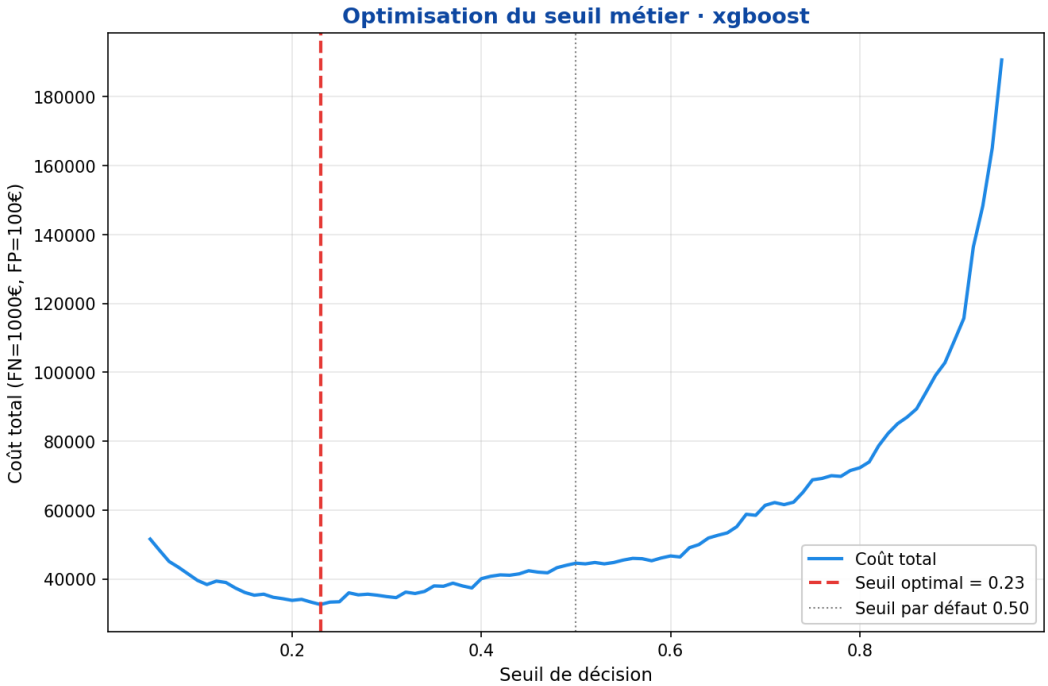


Figure 8 · Coût total en fonction du seuil de décision · le minimum à 0.23 est l'optimum métier. Le seuil par défaut 0.5 est sous-optimal.

Tâches bonus · multi-classe et régression

Multi-classe · XGBoost prédit le type de panne (5 classes) avec macro-F1 = 0.93. **Régression RUL** · Random Forest prédit la durée de vie restante avec MAE = 9.6 h et R^2 = 0.67.

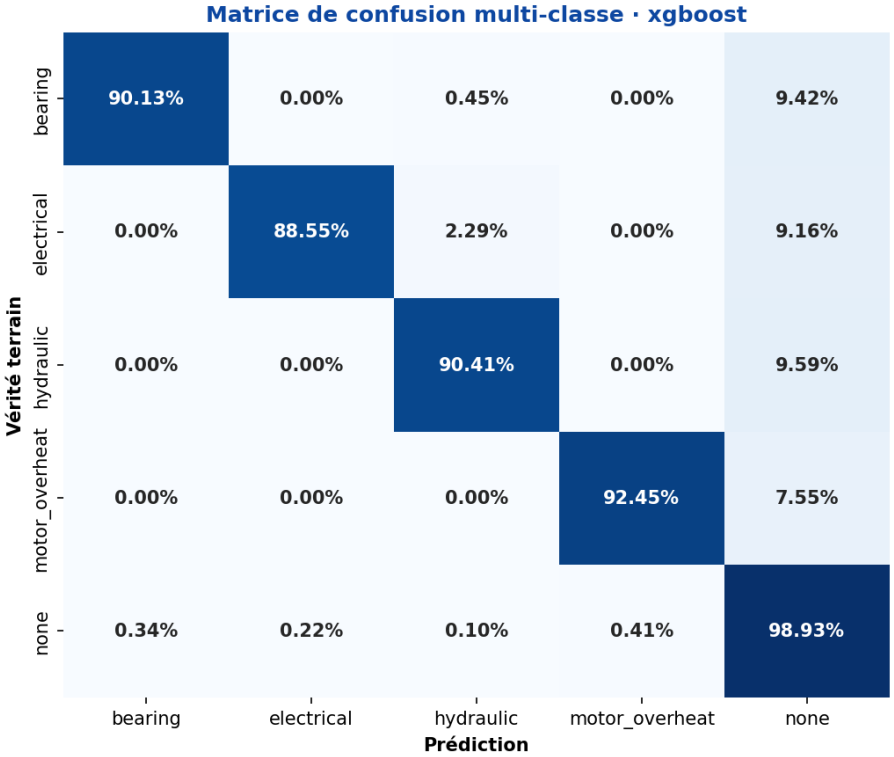


Figure 9 · Matrice de confusion multi-classe · diagonale forte = bonne précision par classe.

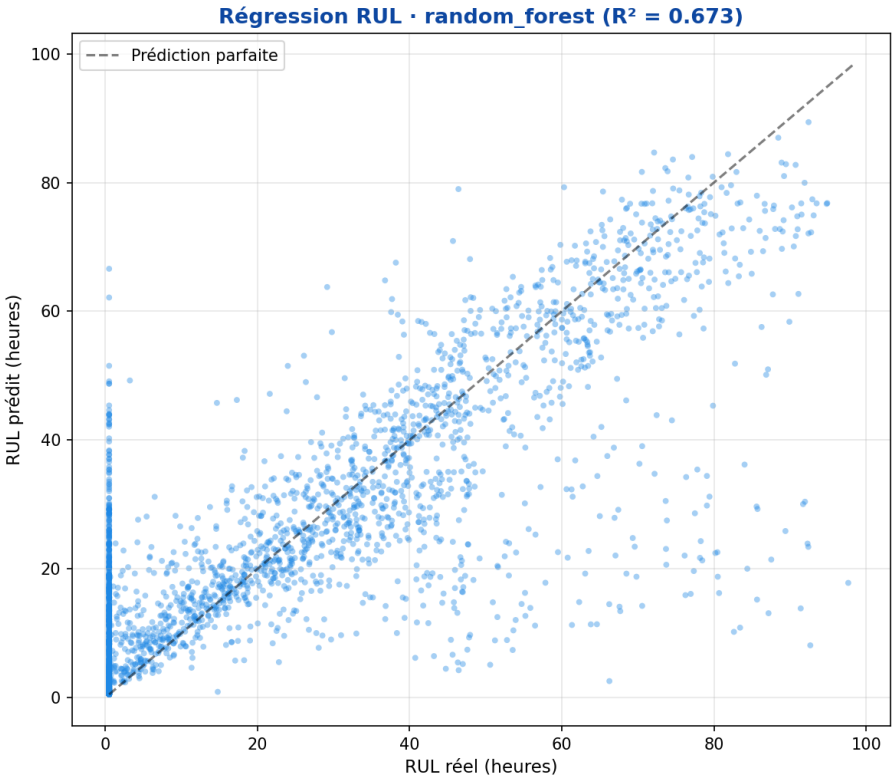


Figure 10 · Régression RUL · prédictions vs valeurs réelles.

Tests automatisés

23 tests pytest couvrent · construction du ColumnTransformer (5 tests · absence de NaN, schéma de sortie, idempotence), factories de modèles (10 tests · binaire, multi-classe, régression), API (4 tests httpx · /predict, /health, /model-info, validation Pydantic), smoke tests d'imports et de chemins (4 tests). Tous passent en ~7 secondes.

Politique stricte · les tests utilisent le CSV Kaggle réel (sample 500 lignes) et skippent proprement si le dataset est absent. Aucun fallback synthétique.

11

Limites et pistes d'amélioration

Une démarche scientifique honnête expose ce qui ne marche pas ou ce qu'on n'a pas eu le temps de faire. Quatre limites majeures et leurs pistes de mitigation pour une v2.

Limite 1 · Dataset simulé

Les données Kaggle sont générées · les corrélations sont « trop propres » comparées à un parc industriel réel où les capteurs déraillent imprévisiblement, les machines sont hétérogènes en âge et en usage, et les modes de défaillance évoluent. **Piste v2** · remplacer par un dataset constructeur réel ou par des données synthétiques générées avec un simulateur physique calibré.

Limite 2 · Pas de validation temporelle

Le split est aléatoire stratifié. Une vraie validation industrielle devrait utiliser une **sliding window cross-validation** qui respecte l'ordre temporel (entraîner sur le passé, tester sur le futur). C'est plus dur car les distributions peuvent dériver dans le temps. **Piste v2** · TimeSeriesSplit de scikit-learn ou expanding window.

Limite 3 · Coûts métier figés

Les coûts FN/FP sont supposés constants (1000 € / 100 €). En réalité ils varient par machine (un robot critique coûte plus qu'un convoyeur secondaire), par criticité (haute saison vs basse saison) et par contexte (équipe disponible ou pas). **Piste v2** · table de référence par machine dans une BDD, recalcul du seuil dynamiquement.

Limite 4 · Pas de drift monitoring

Le modèle est entraîné une fois, déployé, et c'est tout. En production réelle, les distributions de capteurs dérivent (machines vieillissantes, remplacements, nouveaux modes opératoires) · il faut détecter ce **concept drift** et ré-entraîner périodiquement. **Piste v2** · intégration de *Evidently AI* ou *Whylogs* pour le monitoring + déclenchement automatique d'un ré-entraînement quand le drift dépasse un seuil.

Autres pistes v2

- **Streaming temps réel** · Kafka + Spark Streaming sur les flux capteurs
- **MLOps complet** · MLflow tracking, Docker pour l'API, CI/CD GitHub Actions
- **Dashboard temps réel** · WebSocket pour les alertes push aux téléphones
- **Feature engineering avancé** · ratios capteurs, anomaly_trend (FFT)
- **Audit de biais** · performance par sous-groupe (type machine, équipe)

12

Conclusion

Ce que nous avons réalisé

Le projet livre un **système intelligent complet** qui satisfait l'ensemble des exigences du cahier des charges et dépasse plusieurs critères bonus ·

- **4 modèles ML/DL** comparés rigoureusement (Logistic, Random Forest, XGBoost, MLP)
- **3 tâches de prédiction** · binaire principale + multi-classe + régression RUL (bonus)
- **8 scripts numérotés** · pipeline reproductible end-to-end
- **33 figures + 6 métriques** par modèle + cross-validation 5-fold
- **Dashboard vision métier** · 5 onglets pour responsable maintenance
- **API REST FastAPI** · 3 endpoints + Swagger + tests httpx (bonus optionnel)
- **Auto-install** des dépendances · zéro setup au clone
- **23 tests automatisés** · qualité de la base de code

Ce que nous avons appris

Sur le ML supervisé · l'importance d'une EDA rigoureuse avant toute modélisation, le piège du data leakage et la valeur des Pipelines sklearn pour s'en prémunir, le choix des bonnes métriques en classes déséquilibrées (F1 et PR-AUC plutôt qu'accuracy), la valeur d'un score composite F1 - 0.5×σ qui pénalise l'instabilité, l'intérêt du tuning bayésien Optuna face à GridSearch.

Sur l'industrialisation · la séparation modèle/interface, la sérialisation joblib, les contrats d'interface entre dashboard et API, l'importance de la portabilité (chemins absolus, auto-install), la cost-sensitive thresholding qui fait passer un modèle d'académique à opérationnel.

Sur la posture pro · documenter ses choix (ce rapport, les ADR du dossier *docs/*), exposer ses limites honnêtement, séparer la vision data scientist (matrices de confusion) de la vision métier (machines critiques, économies réalisées).

Compétences RNCP40875 Bloc 2 démontrées

C3.1 · Préparation données	ColumnTransformer anti-leakage opérationnel.
C3.2 · Dashboard interactif	Streamlit 5 onglets, Plotly interactif, responsive.
C3.3 · EDA rigoureuse	8 figures + stats descriptives + analyse NaN documentée.
C4.1 · Stratégie d'intégration IA	ROI cas d'usage + ratio coûts FN/FP documenté.
C4.2 · Modèles prédictifs ML/DL	4 modèles dont MLP, F1=0.886 sur test set.

C4.3 · Évaluation + écoresponsabilité	6 métriques + temps train/inférence par modèle.

13

Annexes

A1 · Arborescence du projet

```
maintenance-predictive-industrielle/
■■■ api/                    # FastAPI · 3 endpoints + Swagger
■■■ dashboard/               # Streamlit · 5 onglets vision métier
■■■ data/raw/                # CSV Kaggle (gitignore)
■■■ docs/                    # Documentation + ADR + CONFORMITE_RNCP.md
■■■ models/                  # .joblib sérialisés (gitignore)
■■■ reports/02..05,07-10/    # Sorties par script (figures, métriques)
■■■ reports/06/              # Rapport PDF (le présent document)
■■■ reports/11/              # Présentation PPTX (24 slides)
■■■ scripts/                 # 8 scripts numérotés (pipeline)
■   ■■■ 02_eda.py            # Analyse exploratoire
■   ■■■ 03_train_models.py   # 4 modèles + CV
■   ■■■ 04_interpret.py      # SHAP + permutation
■   ■■■ 05_generate_diagrams.py
■   ■■■ 07_train_multiclass.py    # bonus
■   ■■■ 08_train_regression.py    # bonus
■   ■■■ 09_tune_hyperparams.py    # Optuna
■   ■■■ 10_calibrate.py           # seuil métier
■■■ src/                     # Bibliothèque interne
■   ■■■ bootstrap.py         # Auto-install pip
■   ■■■ config.py            # Constantes centrales
■   ■■■ data_loader.py       # Lecture CSV Kaggle (lecture seule)
■   ■■■ preprocessing.py     # ColumnTransformer
■   ■■■ models.py + models_multiclass.py + models_regression.py
■   ■■■ evaluation.py        # 6 métriques + plotting
■   ■■■ interpretability.py  # SHAP + permutation
■   ■■■ calibration.py       # Reliability + cost-sensitive
■   ■■■ tuning.py            # Optuna TPE
■   ■■■ diagrams.py          # 4 schémas pédagogiques
■■■ tests/                   # pytest · 23 tests
■■■ README.md
■■■ requirements.txt
```

A2 · Conformité RNCP40875 Bloc 2 (matrice complète)

Critère	Intitulé	Preuve
C3.1	Préparer/nettoyer les données	src/preprocessing.py · ColumnTransformer (Imputer + Scaler + OHE)
C3.2	Tableau de bord interactif inclusif	dashboard/app.py · Streamlit 5 onglets · Plotly · responsive

C3.3	Analyse exploratoire de données	scripts/02_eda.py · 8 figures + stats + analyse NaN
C4.1	Stratégie d'intégration de l'IA	Onglet Impact économique · ROI · ratio FN/FP 10:1
C4.2	Modèles prédictifs ML/DL	scripts/03 · 4 modèles dont MLP · F1=0.886 sur test
C4.3	Évaluation + écoresponsabilité	6 métriques + CV 5-fold + temps train/inférence

A3 · Glossaire

EDA	Exploratory Data Analysis · l'étape d'observation des données avant modélisation.
Pipeline	Enchaînement reproductible de transformations · Imputer → Scaler → OHE → Modèle.
Data leakage	Quand l'info du test fuite dans le train. Erreur classique du débutant.
CV	Cross-validation · découper le train en k parts pour évaluer la stabilité.
F1	Moyenne harmonique de Precision et Recall. Métrique principale en déséquilibre.
ROC-AUC	Area Under ROC Curve · qualité globale, peut être trompeuse en déséquilibre.
PR-AUC	Area Under Precision-Recall Curve · plus dure, plus parlante en déséquilibre.
SHAP	SHapley Additive exPlanations · contribution chiffrée de chaque feature.
RUL	Remaining Useful Life · durée de vie restante en heures.
Optuna TPE	Tree-structured Parzen Estimator · sampler bayésien efficient.
Calibration	Qualité des probabilités · un modèle calibré dit 0.7 → 70% de chance réelle.
Cost-sensitive	Optimisation du seuil de décision en fonction des coûts métier asymétriques.
MLOps	Machine Learning Operations · industrialisation du cycle de vie ML.
Drift	Dérive des distributions de données en production · nécessite ré-entraînement.

A4 · Contact et reproductibilité

Adam Beloucif (20220055) · adam.beloucif@efrei.net
Emilien Morice (20241824) · M1 Mastère Data Engineering & IA
EFREI Paris Panthéon-Assas Université · Année universitaire 2025-2026
Repo · github.com/Adam-Blf/maintenance-predictive-industrielle

Reproductibilité · seed unique *RANDOM_STATE=42* propagé partout · auto-install des dépendances pip via *src/bootstrap.py* · chemins absolus calculés depuis *__file__* (portable Win/Linux/macOS) · 23 tests pytest. Le clone du repo + téléchargement Kaggle suffit pour relancer toute la chaîne.